LLM, SLM, or Hybrid Utilization: Decision Factors Across Performance, Efficiency, and Domain — A Systematic Literature Review

Doğukan Topçu, Abdulbaki Çeltik, Berkhan Kargılı, Meriç Kelemeden, Hüseyin Ünlü, and Onur Demirörs

Izmir Institute of Technology, Izmir, Turkey
{dogukantopcu, abdulbakiceltik, berkhankargili, merickelemeden}@std.iyte.edu.tr
{huseyinunlu, onurdemirors}@iyte.edu.tr

Abstract. Large language models (LLMs) deliver strong reasoning and generation performance, but their cost, latency, energy demand, and hardware requirements remain high. Small language models (SLMs) are better suited to edge and privacy-sensitive deployment, yet their capabilities vary across domains. This paper presents a systematic literature review of 54 studies on collaborative SLM–LLM systems spanning hybrid, multi-agent, retrieval-centered, and ensemble architectures, with monolithic LLMs treated as baselines rather than as a primary architecture class. Guided by four research questions on performance, efficiency, orchestration, and domain suitability, we apply a PRISMA 2020-based review design with a three-pass qualitative coding scheme that synthesizes models, architectures, datasets, metrics, latency, cost, energy, and reported limitations. Corpus analysis shows that 70% of studies date to 2025 and hybrid architectures are the most prevalent design; a contribution-type mapping confirms that solution proposals dominate while energy and cost remain markedly underreported relative to accuracy. The literature shows a clear shift toward hybrid and multi-agent designs, but outcome coding reveals that strong results are reliably associated with explicit control layers (routers, retrievers, planners) rather than with model scale. Overall, SLMs are most competitive when paired with routing, retrieval, planning, or fusion mechanisms in narrow, structured, or domain-specific settings.

Keywords: systematic literature review · large language models · small language models · hybrid architectures · multi-agent systems · edge-cloud inference

1 Introduction

Large language models (LLMs) excel at reasoning, coding, and multimodal tasks [?,?,?], but their hardware and deployment requirements often confine them to cloud infrastructure or high-end local systems, raising privacy, cloud dependence, and cost concerns [?,?,?,?].

Small language models (SLMs) are attractive for local deployment and lower-cost, data-sovereign operation. Fine-tuning or distillation can also make them competitive on domain-specific tasks [?,?,?]. They pair naturally with routing, verification, and planning mechanisms [?,?,?]. However, SLMs still trail frontier LLMs on broad reasoning and tasks requiring extensive contextual knowledge [?,?], which has driven recent work on hybrid and agent-based architectures that combine small and large models for a better capability–efficiency trade-off [?,?,?].

This is primarily a systems-design question. Across domains, hybrid and multi-agent architectures aim to improve not only accuracy but also latency, cost, and computational efficiency through edge–cloud verification, expert routing, retrieval, and planner-based orchestration [?,?,?,?,?,?,?]. These developments motivate a closer examination of how collaborative architectures allocate work between small and large models, when monolithic LLM baselines remain necessary, and when orchestration provides the best balance between capability and efficiency.

Despite this growing interest, existing secondary studies do not address this question directly. The only PRISMA-guided predecessor reviews the SLM category broadly rather than collaborative SLM–LLM systems as a primary unit of analysis [?]; narrative surveys offer valuable conceptual breadth but neither treat monolithic LLMs as controlled baselines nor extract latency, cost, and energy as first-class outcome variables [?,?,?]. The 2025–2026 wave of hybrid and multi-agent orchestration architectures is covered only partially by these works, leaving a gap in systematic, traceable evidence about when collaboration outperforms monolithic deployment under realistic engineering constraints.

Accordingly, this review examines when hybrid or multi-agent SLM–LLM architectures provide a better trade-off between capability and efficiency than monolithic LLM baselines, and under which domain conditions routing, retrieval, planning, and role specialization allow smaller models to contribute effectively.

The main contributions are fourfold: (1) a synthesis of 54 studies on hybrid SLM–LLM architectures, retrieval-centered collaboration, ensembles, and multi-agent workflows; (2) a review structure organized around four practical questions on performance, efficiency, orchestration, and domain suitability; (3) a three-pass qualitative coding scheme that links every synthesis claim to specific primary studies, distinguishing directly extracted data from analytical inference and thereby enabling independent verification of conclusions; and (4) a corpus-level characterisation showing that 70% of the included work dates to 2025, that control-layer presence is the most consistent differentiator between high- and low-performing collaborative designs, and that energy remains systematically underreported relative to accuracy across the field.

The remainder of the paper is organized as follows. Section ?? positions this review within prior secondary studies. Section ?? details the search protocol, eligibility criteria, and qualitative coding procedure. Section ?? presents and synthesizes the results across the four research questions. Section ?? discusses threats to validity, and Section ?? concludes.

2 Related Works

Prior secondary studies on small and large language model collaboration can be organized into three groups: a PRISMA-guided systematic literature review, narrative survey papers, and adjacent systematic mapping work. We discuss the contributions of each group on their own terms and then position the present review in terms of scope and methodology.

2.1 Systematic Literature Review

Corradini et al. [?] conduct a PRISMA 2020-guided systematic literature review on the state of SLMs, covering definitions, capability benchmarks, training paradigms (pre-training, instruction tuning, distillation), and early deployment characteristics across diverse architectures and domains. Their study is the only formally protocol-driven predecessor work; it provides a rigorous and reproducible characterization of the SLM landscape as a whole. However, its scope is intentionally broad with respect to the SLM category in general: it does not focus on collaborative SLM–LLM systems as a primary unit of analysis, it does not treat monolithic LLMs as controlled baselines, and it provides limited coverage of the 2025–2026 wave of hybrid and multi-agent orchestration architectures that forms the core of the present corpus. Engineering-level efficiency metrics—latency, cost, and energy—are not extracted as primary outcome variables.

2.2 Narrative Surveys

Three recent surveys address aspects of SLM–LLM collaboration from complementary perspectives [?,?,?]. Wang et al. [?] provide a comprehensive taxonomy of SLM architectures and training strategies, including fine-tuning, compression, and distillation, with attention to deployment characteristics and trustworthiness. Chen et al. [?] survey collaboration mechanisms between large and small models, distinguishing distillation pipelines, routing strategies, ensemble methods, and emerging agent-based coordination patterns, and identify key design principles for leveraging complementary model strengths. Wang et al. [?] focus on performance dimensions of SLM–LLM collaboration, reviewing benchmark results across reasoning, coding, and domain-adaptation tasks, and examining cost-effectiveness and cloud–edge privacy trade-offs. These surveys offer valuable breadth and conceptual synthesis that a narrower protocol-driven review cannot replicate. They differ from protocol-based reviews primarily in the traceability of corpus construction: inclusion rationale is not systematically documented through a registered screening procedure, which limits the ability to independently verify corpus completeness—a distinction of transparency rather than of quality.

2.3 Methodological Positioning

A range of methodological frameworks is available for secondary studies in software and systems engineering. Kitchenham and Charters [?] formalize systematic review guidelines with emphasis on structured evidence synthesis and quality appraisal. Petersen et al. [?] introduce the systematic mapping study approach, which aims to characterize a field structurally through publication frequency, research type classification, and topic coverage analysis. Informal narrative synthesis, as employed by the surveys above, serves distinct objectives and produces broader conceptual coverage. PRISMA 2020 [?] was selected for the present study for three reasons specific to the research objectives: (i) the target architecture class—collaborative SLM–LLM systems—requires multi-criterion judgment at screening to separate from adjacent literature, making documented stage-by-stage inclusion decisions essential; (ii) the PRISMA flow diagram and eligibility matrix make corpus construction independently verifiable; and (iii) PRISMA's structured reporting requirements are well-suited to drawing operational conclusions from heterogeneous empirical evidence. This choice is complementary to, not superior to, the Kitchenham-guided or narrative traditions employed by related works; each tradition serves different secondary-study objectives.

2.4 Scope Differentiation

The present review extends beyond existing secondary work in four respects. First, **architectural scope**: the review is narrowed exclusively to collaborative SLM–LLM systems (hybrid, multi-agent, retrieval-augmented, and ensemble architectures), with monolithic LLMs treated only as controlled baselines; related surveys treat SLMs more broadly or include studies without a collaborative component. Second, **temporal coverage**: 98% of included studies were published in 2024–2026, capturing recent hybrid edge–cloud and multi-agent orchestration developments that postdate most related surveys. Third, **engineering orientation**: latency, cost, and energy are extracted as primary outcome variables alongside accuracy, making efficiency trade-offs explicit first-class research questions; no related survey reports all four metrics systematically across a documented study corpus. Fourth, **evidence traceability**: a three-pass qualitative coding scheme (Section **??**) links each synthesis claim to specific primary study identifiers, and a cross-study RQ-to-evidence matrix (Table **??**) supports independent verification of conclusions following the concept-centric synthesis approach advocated by Webster and Watson [?]. Table **??** summarises these four differentiators at a glance.

3 Research Methodology

In this section we will outline the review protocol, including the research questions, search strategy, eligibility criteria, study selection, and data extraction and synthesis process.

Table 1. Scope differentiation of this review versus prior secondary work.

Prior work	Key scope limitation	Present review adds
Corradini et al. [?]	SLMs reviewed broadly; collaborative SLM–LLM systems not the primary unit; efficiency metrics not extracted.	Collaborative systems as primary unit; latency, cost, and energy as first-class outcomes.
Wang et al. [?]	Narrative survey; no documented inclusion screening.	PRISMA 2020 stage-by-stage screening; independently verifiable corpus.
Chen et al. [?]	No temporal coverage of 2025–2026 architectures; narrative only.	98% corpus from 2024–2026; protocol-driven inclusion.
Wang et al. [?]	No cross-study RQ-to-evidence matrix; no provenance annotation.	S1–S54 evidence traceability; *[Synthesized]* provenance labels; concept-centric RQ matrix.

3.1 Research Questions

To systematically evaluate the technical and operational trade-offs between collaborative SLM–LLM systems and monolithic LLM baselines, the review protocol is structured around the following four core research questions (RQs):

- **RQ1 (Performance):** How does the performance (accuracy/efficacy) of multi-agent SLMs or hybrid architectures (LLM + SLM) compare with that of monolithic LLM baselines in specific reasoning and classification tasks?
- **RQ2 (Efficiency):** What is the impact of hybrid and multi-agent architectures on computational efficiency, specifically regarding inference cost, latency, and energy consumption, compared to monolithic LLM baselines or full-LLM inference?
- **RQ3 (Orchestration):** What orchestration and routing mechanisms are employed to manage the division of labor between large and small models in collaborative systems?
- **RQ4 (Domain Suitability):** In which specific domains (e.g., healthcare, coding, telecommunications, and edge or multimodal settings) do collaborative SLM-centered architectures achieve performance comparable to monolithic LLM baselines, and where do they fail?

3.2 Information Sources and Search Strategy

To capture a comprehensive and high-quality corpus of contemporary collaborative language model architectures, the literature search was centered on the Web of Science (WoS) Core Collection, with the final search performed on 8 February 2026. Scopus and the ACM Digital Library were also queried with the same four-component structure as coverage validation; deduplicated records from both sources were screened against the eligibility criteria, and no additional studies were identified beyond those already captured by the WoS search. The WoS-centric approach and its implications for coverage are discussed under external validity (Section **??**).

The search strategy was systematically developed by extracting the core concepts from the defined research questions and refining the terminology through initial pilot searches. This iterative process helped identify common synonyms and alternate phrasing used in the literature (e.g., "Compact Model" for SLMs, or "Agentic" for multi-agent systems). To guarantee that every retrieved study examined the intersection of model size, collaborative architectural design, and system evaluation, the final query was structured into four mandatory components and applied to the title, abstract, and keyword fields: *("Large Language Model*" OR "LLM" OR "Generative AI" OR "Foundation Model*") AND ("Small Language Model*" OR "SLM" OR "Compact Model*" OR "Lightweight Model*") AND ("Multi-agent" OR "Hybrid" OR "Orchestration" OR "Routing" OR "Ensemble" OR "Agentic") AND ("Performance" OR "Accuracy" OR "Efficiency" OR "Latency" OR "Cost" OR "Benchmark").*

Because the review targets collaborative SLM–LLM systems, all four components were mandatory in the search string. Monolithic LLMs were considered only as baselines reported inside the included studies, not as standalone primary studies.

3.3 Eligibility Criteria

To systematically isolate systems-level engineering research from the broader generative AI literature, retrieved records were evaluated against strict eligibility criteria. A comprehensive breakdown of the inclusion and exclusion parameters applied during the screening phases is presented in Table **??**.

3.4 Study Selection Process

The study selection process was guided by the PRISMA 2020 guidelines [**?**] and is summarised in Figure **??**. The initial search yielded 116 records.

During Identification and Screening, the title, abstract, and keywords of all records were reviewed manually, excluding 36 records that failed the primary eligibility criteria, including non-technical narrative surveys, extended abstracts without full empirical validation, and studies in unrelated domains. The remaining 80 reports advanced to full-text assessment, where 26 additional records were excluded due to model size or architecture mismatch (n=18), topic or scope mismatch (n=5), or lack of original empirical data or a remaining narrative-survey character (n=3). To improve objectivity, title–abstract screening was performed independently by two reviewers before any discussion; initial pairwise agreement across both screening stages reached 93.1% (108 of 116 records; 8 records required consensus discussion). Disagreements were resolved through structured

Table 2. Inclusion and Exclusion Criteria for Study Selection

Criterion	Include	Exclude
Architecture Scope	Collaborative SLM–LLM systems using routing, retrieval, verification, fallback, or role specialization.	SLM-only papers or monolithic LLM comparisons without a collaborative SLM component; monolithic LLMs were allowed only as baselines inside otherwise eligible studies.
Model Size and Transparency	Identifiable SLM component with reported or inferable size not exceeding 7B parameters — a threshold that marks the practical deployment boundary for single-GPU inference without quantization and is consistent with SLM definitions used in related secondary studies [**?**,**?**]; closed-box LLMs allowed only as baseline, teacher, or fallback components.	Primary architecture depends on closed-box/API-bound models with undisclosed details, or no reproducible SLM component can be identified.
Topic Focus	Reports system-level metrics such as performance, latency, memory, cost, or orchestration efficiency.	Focuses only on ethical, sociological, or legal issues without technical benchmarks.
Data Quality	Contains empirical results and experimental validation using benchmarks or custom datasets.	Purely theoretical papers or non-empirical narrative surveys.

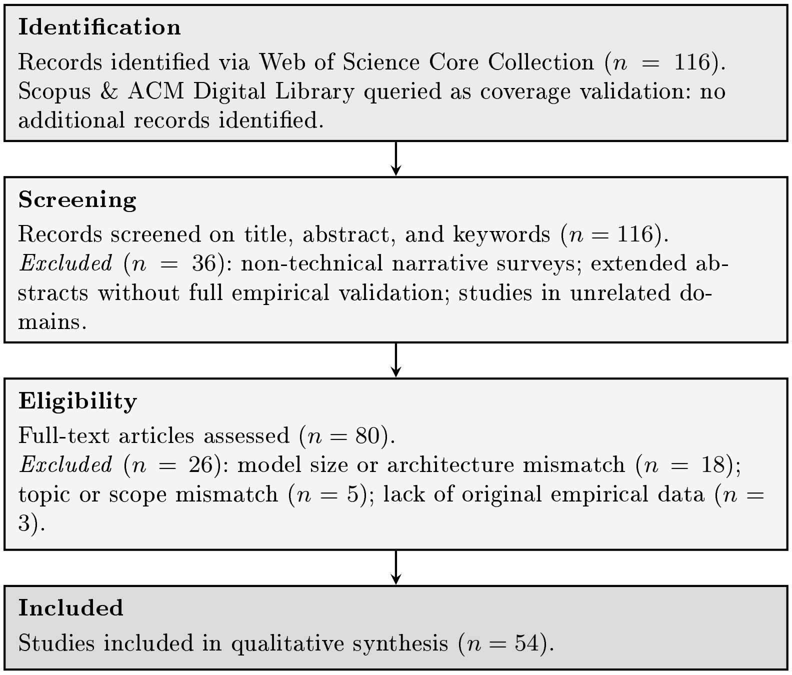

Fig. 1. PRISMA 2020 flow diagram of the study selection process.

consensus discussion, and unresolved cases were escalated to the academic advisors. This procedure was applied at both screening stages, resulting in a final corpus of 54 primary studies.

3.5 Data Extraction and Synthesis Strategy

To systematically address the research questions, a standardized data extraction protocol was applied to the final corpus of 54 primary studies. For each study, data was extracted into a structured matrix to capture both the functional design of the systems and the empirical evidence used to justify them.

The extracted fields included the specific SLM and LLM models utilized; the overarching architecture type (e.g., Hybrid, Multi-Agent, Retrieval-Augmented Generation (RAG), Ensemble); orchestration mechanisms; evaluation metrics; baseline models; and quantitative performance indicators (accuracy, latency, cost, and energy). To ensure analytical rigor, each extracted variable was explicitly mapped to one or more of the core Research Questions (RQs). This mapping, together with concise field definitions, is detailed in Table **??**.

The full comparative analysis table and the comprehensive data extraction spreadsheet are available at `https://bit.ly/SLM-LLM-Data-2026`. The Study ID to citation mapping (S1–S54) is provided in Appendix **??** for cross-referencing purposes. Following data extraction, a narrative synthesis approach was utilized.

Table 3. RQ × data extraction matrix. Check marks indicate direct support; Arch. = architecture, Orchestr. = orchestration, and Base. = baseline.

Field	Description	RQ1	RQ2	RQ3	RQ4
SLM(s)	Specific small language model(s) evaluated, fine-tuned, or deployed.	✓			✓
LLM(s)	Large language model(s) used as a baseline, teacher, or fallback.	✓			✓
Arch. Type	Overall system design (e.g., hybrid, multi-agent, RAG, ensemble).	✓			✓
Orchestr. Method	Coordination mechanism such as routing, decomposition, fusion, or voting.			✓	✓
Agent Count	Number of distinct agents, experts, or role-specific components.		✓	✓	✓
Task Type	Downstream application, domain, or problem setting evaluated.	✓		✓	✓
Dataset	Benchmark, open-source corpus, or proprietary data used in evaluation.	✓		✓	
Metrics	Evaluation criteria used to assess performance and efficiency.	✓	✓	✓	✓
LLM Base.	Standalone performance score of the comparison large model.	✓		✓	✓
SLM Base.	Standalone performance score of the comparison small model.	✓		✓	✓
Accuracy	Final performance score achieved by the proposed architecture.	✓		✓	
Latency (ms)	Reported inference delay, throughput, or time-to-response.		✓	✓	✓
Cost ($)	Reported API cost or computational savings versus full LLM use.		✓	✓	✓
Energy	Power use, joules-per-token, or carbon-related efficiency metric.		✓	✓	✓
Findings	Main study takeaway on SLM–LLM collaboration or orchestration.	✓	✓	✓	✓
Limits	Reported constraints, failure modes, or deployment bottlenecks.		✓	✓	

Because the included studies operate across diverse domains (e.g., telecommunications, healthcare, software engineering) and report varying metrics, a purely statistical meta-analysis was not feasible. Instead, studies were grouped by their architectural taxonomy, allowing for a comparative analysis of how different orchestration methods and domain constraints impact the performance and efficiency trade-offs between large and small models.

3.6 Synthesis Protocol and Qualitative Coding

To ensure that synthesis claims are systematically derived from and traceable to primary evidence, data extraction was followed by a three-pass qualitative coding procedure [**?**]. This procedure standardizes interpretive synthesis decisions across heterogeneous studies and classifies the provenance of extracted statements to distinguish direct evidence from analytical inference.

First pass—Architecture and Orchestration Categorization. Each study was independently coded by two reviewers for (i) primary architecture type: Hybrid, Multi-Agent, Retrieval-Augmented, Ensemble, or Combined; and (ii) dominant orchestration pattern: token-level verification, query-level routing, task decomposition, retrieval augmentation, multimodal fusion, or role specialization. Inter-rater disagreements were resolved through structured discussion, following the same consensus procedure applied during screening (Section **??**), with escalation to academic advisors for unresolved cases.

Second pass—Domain and Metric Coverage Coding. Each study was classified by primary application domain (Healthcare/Medical, Telecommunications/Networks, Software Engineering, Education/Assessment, Industrial/Edge/IoT, or General NLP) and by metric availability: (a) performance-only; (b) performance plus at least one efficiency metric; or (c) full-metric reporting including accuracy, latency, cost, and energy. This coding produced the distribution statistics reported in Section **??**.

Third pass—Performance Outcome and Evidence Provenance Classification. Each study's primary empirical finding was classified as one of: (a) outperforms monolithic SLM baseline; (b) matches monolithic LLM baseline; (c) outperforms LLM baseline; or (d) mixed or task-dependent results. Concurrently, each extracted data item was assigned a provenance label:

– *Direct numerical extraction*: a quantitative result reproduced verbatim from the primary paper;
– *Direct textual extraction*: a qualitative claim or limitation reproduced verbatim or with minor paraphrase, with page reference;
– *Analytical synthesis*: a pattern, limitation, or implication derived by the review team from architectural descriptions or indirect evidence in the source paper, where the primary study does not make the statement explicitly.

In the supplementary extraction spreadsheet (`https://bit.ly/SLM-LLM-Data-2026`), cells derived by analytical synthesis are annotated "[Synthesized]" to distinguish them from directly extracted claims. This distinction is particularly relevant for limitation statements: where an author explicitly states that a metric was not measured, the limitation is recorded as a direct textual extraction; where a limitation is inferred by the reviewers from architectural properties not discussed by the authors, it is recorded as an analytical synthesis. For example, in S1 the latency field records "not reported numerically" as a direct textual extraction, while the associated limitation note regarding production-time overhead from iterative refinement loops is recorded as an analytical synthesis based on the described pipeline.

Following Webster and Watson [?], synthesis conclusions in Section **??** are organized concept-centrically rather than study-by-study: each concept (orchestration pattern, efficiency trade-off, domain constraint) is linked to the studies providing evidence for it via the S1–S54 identifier scheme. Design pattern claims in Section **??** were retained as primary implications only when supported by at least four independent primary studies reporting consistent positive outcomes under comparable operational constraints; the supporting study set for each pattern is stated explicitly in that section.

4 Findings

Table **??** provides a one-row-per-RQ orientation of the principal findings before the detailed analysis.

Table 4. Executive summary of findings per research question.

RQ	Focus	Principal finding	Evidence
RQ1	Performance	Hybrid and multi-agent architectures match or exceed LLM baselines in structured, domain-specific settings when an explicit routing or decomposition layer is present.	39/54 studies
RQ2	Efficiency	Token-level routing achieves >80% cost reduction; orchestration overhead polarizes latency; energy is the least-reported metric across the corpus.	47/54 studies
RQ3	Orchestration	Five recurring mechanism families—token verification, query routing, RAG, task decomposition, and multimodal fusion—emerge; selection is constraint-driven.	54/54 studies
RQ4	Domain suitability	Healthcare, telecom, edge/IoT, and code tasks yield the strongest gains; open-ended generation tasks show the least consistent improvement.	54/54 studies

The 54 included studies cluster into four recurring architectural patterns. Table **??** maps each pattern to its dominant orchestration mechanisms, the engineering constraints it addresses, and the primary studies assigned to it. This concept matrix provides the organizing structure for the research-question-based analysis that follows.

Table 5. SLM–LLM architectural concept matrix.

Architectural Pattern	**Orchestration Mechanism**	**Key Engineering Constraints**	**Primary Studies**
Hybrid Architectures	Token-level routing, draft-verification, uncertainty-aware skipping, and edge–cloud distribution.	API cost, edge–cloud bandwidth, and latency thresholds.	S2, S13, S27, S32, S33, S45, S46, S48, S49, S53.
Multi-Agent Systems	Role specialization, iterative verification, adversarial debate, and feedback loops.	Reasoning depth, logical accuracy, and complex task decomposition.	S1, S6, S7, S11, S15, S16, S20, S29, S37, S43, S51, S52, S54.
Retrieval-Centered (RAG)	Adaptive retrieval planning, context expansion, fact-grounding, and action-space constraint.	Hallucination reduction, data privacy, and strict domain accuracy.	S4, S10, S21, S30, S31, S44.
Ensembles and Heterogeneous Fusion	Tabular transformation, cross-modal late fusion, and prompt-guided multimodal sampling.	Hardware limits on CPUs and edge devices, and non-text modality processing.	S3, S5, S8, S9, S12, S14, S17–S19, S22–S26, S28, S34–S36, S38–S42, S47, S50.

4.1 Study Characteristics and Corpus Overview

Before addressing the research questions, a characterization of the 54-study corpus provides context for interpreting the synthesis.

Temporal distribution. The corpus is concentrated in recent years: one study dates to 2023 (2%), 13 to 2024 (24%), 38 to 2025 (70%), and two to early 2026 (4%). The dominance of 2025 publications indicates an emergent and fast-moving research area in which collaborative SLM–LLM architectures are still consolidating. Publications cluster around two conference cycles (ACL/EMNLP and AAAI/WWW) and several open-access engineering journals, with a notable share appearing first as arXiv preprints later published in conference or journal proceedings. Figure **??** illustrates this sharp temporal acceleration.

Fig. 2. Publication-year distribution of the 54 included primary studies.

Architecture distribution. Hybrid architectures—comprising token-level verification, query-level routing, and task-decomposition systems—constitute the largest group (27 studies), followed by multi-agent systems (18 studies), retrieval-augmented designs (12 studies), and ensemble approaches (6 studies). Because several studies combine multiple patterns (e.g., multi-agent systems that also employ RAG), per-study architecture labels are not mutually exclusive and aggregate counts exceed 54; the full per-study coding is available in the supplementary spreadsheet. Hybrid designs grew as a proportion of the corpus between 2024 and 2025, while ensemble-only studies remained stable. Multi-agent architectures increased substantially in 2025, consistent with the broader uptake of agentic frameworks in the literature.

Domain distribution. Healthcare and clinical applications account for the largest single domain (14 studies), spanning radiology labeling, clinical summarization, TCM classification, and biomedical agents. Software engineering and code-related tasks form the second-largest group (8 studies), covering code correction, static analysis, comment generation, and robot introspection. Telecommunications and network optimization (6 studies), edge/IoT/industrial settings (6 studies), and education/assessment tasks (4 studies) follow. The remaining 16 studies address general NLP tasks or cross-domain benchmarks. Domain distribution is uneven; healthcare and NLP are substantially better represented than education, transportation, and emerging multimodal settings. The breakdown is visualised in Figure **??**.

Fig. 3. Applied-domain distribution of the 54 included studies (n per slice shown in legend).

Metric reporting. The second coding pass (Section **??**) confirmed that performance metrics (accuracy, F1, task-specific scores) are reported in nearly all 54 studies. Latency is reported in approximately one-third of the corpus, cost in fewer than one-quarter, and direct energy measurement in only a handful of studies. This uneven reporting profile reflects the field's current maturity and directly constrains what cross-study efficiency comparisons are possible; this limitation is discussed in full under RQ2 (Section **??**).

Contribution types. Classified following Petersen et al. [**?**], the corpus is dominated by *solution proposals* (42 studies presenting novel systems with experimental evaluation), with a smaller set of *evaluation studies* (9 studies providing comparative analyses of existing models or frameworks) and a few *validation studies* (3 proof-of-concept designs without full benchmark evaluation). The predominance of solution proposals is consistent with the field's early stage: few standardized benchmarks exist for collaborative SLM–LLM systems, and most papers define their own evaluation protocols.

4.2 RQ1: Comparative Performance of Hybrid and Multi-Agent Systems

The review shows that both hybrid and multi-agent architectures can outperform standalone small-model baselines, but their advantages emerge through different forms of coordination rather than parameter scaling alone. Hybrid systems achieve their strongest gains when computation is selectively distributed between local small models and stronger remote models. Token-level edge–cloud collaboration improves small-model performance without requiring full-query escalation: draft-verification, low-confidence routing, and uncertainty-aware skipping allow small models to approach LLM-level quality while invoking the larger model selectively (S13, S48, S49). At the sub-task level, structured task decomposition and cloud–edge allocation also maintain reasoning accuracy comparable to the best baselines (S14). In domain-specific settings, hybrid orchestration can outperform larger general-purpose models: a fine-tuned Phi-2 (2.7B) combined with retrieval, reranking, hybrid search, and long-context support achieves 84.20% accuracy, exceeding GPT-4o mini (82.05%) and GPT-4o (80.80%) on telecom question answering (S17).

Multi-agent systems, by contrast, show their comparative strength when performance depends on role specialization, critique, verification, and distributed reasoning. A three-agent Phi-3-based architecture performs on par with human experts in conceptual genomics tasks, although its gains weaken on more complex code-generation workflows, indicating that multi-agent benefits remain task-sensitive (S52). In reasoning-intensive question answering, a Proponent–Opponent–Arbitrator structure built on compact models such as Qwen2.5 0.5B and DeepSeek-R1 1.5B surpasses both single-agent reasoning and vanilla LLM baselines (S43). Likewise, assistant–checker collaboration supported by memory and feedback loops enables small-model multi-agent systems to approach or exceed larger baselines across database, knowledge-graph, and code-correction tasks (S29).

Summary (RQ1). Hybrid and multi-agent architectures consistently outperform monolithic SLM baselines and can match or exceed LLM baselines in narrower settings. The performance advantage is driven by orchestration strategy—selective escalation, role decomposition, verification loops—rather than by model size alone. Gains are strongest in structured, domain-specific tasks and weakest in open-ended reasoning that requires broad contextual knowledge.

Key takeaways.

- Hybrid systems are strongest when selective escalation balances model quality against latency and cost.
- Multi-agent systems benefit tasks that require role specialization, critique, and iterative verification, but their gains remain task-sensitive.
- Orchestration strategy explains comparative performance more consistently than model size alone.

4.3 RQ2: Efficiency, Cost, Latency, and Energy Trade-offs

Efficiency motivates many hybrid and multi-agent systems, but the impact on cost, latency, and energy depends heavily on orchestration. Hybridization is not a universal efficiency solution; it often shifts bottlenecks rather than eliminating them.

Regarding inference cost and token consumption, architectures employing selective escalation or task decomposition drastically reduce reliance on massive standalone LLMs. S49 achieves an over 80% cost reduction compared to full LLM inference by routing fewer than 7% of generated tokens to the cloud. Similarly, S13 uses a token-level draft-verification method to match the quality of a 70B parameter LLM while consuming only 25.8% to 31.2% of standard cloud inference costs. Query-level decomposition yields similar savings: S14 cuts average API costs by 83.57% via parallel reasoning. Computational savings are also heavily emphasized through token reduction. S40 achieves a 95% reduction in LLM token transmissions via federated uncertainty-aware routing, and S15 eliminates redundant multi-round communications through an SLM-generated directed acyclic graph, reducing token consumption by 35% to 70%. In dialogue state tracking, S16 highlights that a retrieval-based router reduces overall computational FLOPs by approximately 62%.

Conversely, the impact on latency is highly polarized. In optimal scenarios, relying on lightweight models accelerates throughput: a fine-tuned SLM in a hybrid classification pipeline (S10) operates at 7.41 ms on a GPU, and uncertainty-aware skipping in an edge–cloud setup provides 2.54× faster token throughput (S48). However, orchestration mechanisms frequently introduce new bottlenecks. Planners and retrievers add temporal overhead, as shown by the adaptive retrieval planner in S11 (1.42 seconds versus 0.95 seconds for naive retrieval) and by cross-encoder reranking in S17. Multi-agent collaboration also multiplies inference cycles: S43 reports a 1.8× inference-time overhead for adversarial debate, while complex agentic workflows can push average response latency to 107 seconds (S31). Processing time is further affected by hardware and modality constraints, including constrained local hardware, dual-path video sampling, CPU-only ensembles, and key-value cache recomputation at the edge (S20, S23, S50, S5).

Energy consumption remains the least documented metric in the reviewed studies, and most reported efficiency gains are resource proxies rather than direct energy measurements. The clearest energy-centered evidence comes from S4, which argues for energy-per-token as a more informative metric and shows that forcing a small model to execute complex reasoning workflows, such as Chain-of-Thought, can substantially increase its per-query energy use. In some settings, a larger model prompted more directly is therefore more energy-efficient than a smaller model that generates a long reasoning trace. By contrast, S29 reports a 26.4% reduction in computational redundancy through dual-scale memory, which may indicate lower resource demand but does not by itself constitute a direct energy benchmark. Taken together, the review suggests that energy efficiency depends on orchestration and generation length as much as on model size, and that stronger claims require standardized, hardware-specific measurement protocols.

Summary (RQ2). The strongest efficiency gains—often exceeding 80% cost reduction—come from token-level routing and selective escalation. Latency effects are polarized: local SLM inference is fast, but orchestration layers (retrievers, planners, multi-agent cycles) frequently introduce overhead that negates throughput gains. Energy is the least reported metric, and available evidence shows that generation length and reasoning depth matter as much as model size for per-query energy use.

Key takeaways.

- Selective routing and task decomposition can substantially reduce API use, token transmission, and computational demand.
- Latency gains are conditional: routing can accelerate inference, while retrieval, planning, and multi-agent loops can introduce significant overhead.
- Energy claims remain weak without direct, hardware-specific measurements that account for generation length and orchestration cost.

4.4 RQ3: Orchestration and Routing Mechanisms

The studies contain a broad set of orchestration mechanisms that can be organized into five recurring families: *token-level verification*, *query-level routing*, *retrieval augmentation (RAG)*, *task decomposition*, and *multimodal fusion*. Each family addresses a distinct allocation problem—confidence management, difficulty separation, context grounding, subtask specialization, and modality integration—and the choice among them is largely determined by the deployment constraints and data modalities of the target domain. Across the included studies, performance differences are explained less by absolute model size than by how effectively the system retrieves evidence, routes uncertainty, and divides labor across components.

Token-level orchestration is most visible in verification-oriented hybrid systems. S13 uses the SLM as a drafter and the LLM as a verifier on difficult tokens. S48 extends this pattern with uncertainty-aware skipping so that likely acceptable SLM tokens can bypass the remote model. S49 reaches a related design point by routing only low-confidence tokens for external inference. Taken together, these studies show that token-level collaboration provides fine-grained control over answer quality, but it can also introduce communication and synchronization overhead when escalation rates increase.

Query-level routing appears when the system selects a processing path for the full input rather than for individual tokens. S16 exemplifies retrieval-based expert routing at the dialogue level, while S10 shows that instance-level model selection can also be effective in customer-review analysis. These designs are most useful when "easy" and "hard" cases can be separated in a semantically meaningful way before full generation begins.

RAG is a major control mechanism across the included studies even when the architecture is classified as hybrid rather than purely RAG. S17, S11, S39, and S47 use retrieval not only to expand context but also to constrain the action space of a smaller model. In these studies, retrieval acts as a coordination layer that narrows ambiguity, grounds the answer in domain evidence, and reduces reliance on raw model scale alone.

Task decomposition is increasingly important in the recent literature. S14 uses a task decomposer, dependency graph, and plug-and-play adapter to allocate subtasks between local and remote models. S53 uses a feedback-integrated workflow that gathers structured feedback from multiple perspectives before revising the prompt of the smaller model. Related role-based decomposition also appears in S23, S29, and S52. Across these studies, the common idea is that small models remain viable when the workflow can be partitioned into specialist stages rather than handled as a single undifferentiated generation step.

Orchestration frequently extends beyond unimodal text processing to integrate diverse data modalities and specialized neural architectures. Several frameworks are designed to fuse language models with non-text representations and structured data. For example, S20 combines coarse-grained and fine-grained visual semantics through prompt-guided video sampling, while S8 fuses language-model embeddings with graph-convolutional features for molecular-property prediction. This heterogeneous routing is particularly prevalent in applied domains such as medicine; S19 orchestrates convolutional networks (ConvNeXt) for anatomical localization alongside a self-reflective LLM agent, and S34 employs a late-fusion approach to combine structured gradient boosting models (EHR tabular data) with unstructured clinical text analysis. In these settings, orchestration operates as a structured fusion mechanism, ensuring that visual, spatial, or mathematical data is processed by the optimal specialized architecture before language generation occurs.

Summary (RQ3). Orchestration mechanisms cluster into five recurring families: token-level verification, query-level routing, RAG, task decomposition, and multimodal fusion. Each family addresses a distinct allocation problem—confidence management, difficulty separation, context grounding, subtask specialization, and modality integration—and the choice among them is largely determined by the deployment constraints and data modalities of the target domain.

Key takeaways.

- Token- and query-level routing provide different levels of control over when a larger model is invoked.
- Retrieval acts as a control mechanism by grounding responses and constraining the smaller model's action space.
- Task decomposition and heterogeneous fusion enable specialist components to collaborate, but add coordination and synchronization costs.

4.5 RQ4: Domain Suitability and Application-Specific Constraints

The choice between hybrid and multi-agent systems is rarely domain-agnostic. Privacy mandates, latency budgets, and verification demands largely determine the orchestration method. Across the 54 primary studies, distinct architectural preferences emerge across four core applied domains.

In fields with strict privacy constraints or high-stakes accuracy requirements, hybridization is driven by the need to keep data local while grounding the SLM in verifiable facts. Local SLMs fine-tuned on clinical corpora and augmented by RAG are therefore well suited to medical reporting and clinical classification tasks (S19, S30), while few-shot open-source models can be competitive for structured radiology labeling (S12). Related healthcare studies also show that local ensembles, tabular transformations, and distillation can support diagnostic or summarization workflows without relying on a fully remote LLM (S28, S33, S41). In scientific settings, structurally augmented SLMs can be effective when language representations are fused with domain-specific models, as in molecular property prediction with graph-convolutional features (S8).

Tasks requiring rigid syntax, iterative refinement, and logical debugging benefit from multi-agent or feedback-driven SLMs rather than zero-shot massive LLMs. Pedagogy, assessment, code correction, and database tasks favor workflows in which small models receive structure, memory, or explicit feedback (S6, S29, S32). In software and bioinformatics settings, role-specific agents can mimic developer-reviewer workflows and generate or revise code offline, although broad static analysis still favors larger models when the task requires open-ended type inference (S24, S2, S52, S53).

Telecommunications architectures prioritize latency budgets, bandwidth allocation, and domain-specific vernacular, making hybrid edge–cloud systems and retrieval-grounded SLMs the dominant pattern. Telecom QA benefits from fine-tuning, retrieval, reranking, and long-context support because these mechanisms ground smaller generators in technical standards and network terminology (S17, S47). Beyond QA, latency optimization and bandwidth reduction drive hybridization through lightweight prediction, uncertainty-aware skipping, and sub-task decomposition (S37, S48, S14).

In edge, cyber-physical, and multimodal settings, orchestration is governed by hardware limits and input modality. Mission-critical aerospace and geospatial workflows rely on local or distributed agents to process operational data under deployment constraints (S23, S45). Industrial video monitoring and cross-modal sensing benefit from heterogeneous pipelines in which local models handle coarse or sensor-side processing while larger multimodal models are invoked only for complex cases (S20, S1). Overall, these domains show that collaborative SLM–LLM systems are strongest when the workflow can be partitioned according to privacy, latency, modality, or verification needs.

Where collaborative systems underperform. Several conditions consistently produce weak or mixed results. Open-ended reasoning tasks requiring broad contextual knowledge remain the most reliable failure mode: studies involving general-domain question answering without retrieval grounding and tasks demanding multi-hop inference over unseen entities show the smallest gains and most task-dependent outcomes (S3, S12). Broad static analysis, in which type inference and cross-file dependency tracking require model-level generalization, still favours larger monolithic models even when multi-agent wrappers are added (S2, S24). In multi-agent settings, performance degrades when role decomposition is applied to tasks that do not admit natural subtask boundaries: S52 reports that gains on conceptual genomics tasks weaken substantially on complex code-generation workflows that resist modular partitioning. These failure cases share a common pattern: orchestration overhead (retrieval latency, inter-agent communication cycles, planner invocation) exceeds the benefit when the task cannot be partitioned into specialist stages with well-defined inputs and outputs.

Summary (RQ4). Domain-specific constraints—privacy mandates in healthcare, latency budgets in telecom, hardware limits in edge deployments, and syntax rigidity in code tasks—are the primary drivers of architecture selection. Collaborative SLM–LLM systems deliver their strongest results when the workflow can be partitioned to align each component with the constraint it best addresses; they underperform when tasks require broad, undifferentiated generation that cannot be decomposed or grounded.

Key takeaways.

- Healthcare and privacy-sensitive domains favor local, retrieval-grounded models with verifiable evidence.
- Software, reasoning, and telecommunications tasks benefit from structured feedback, specialist agents, or latency-aware edge–cloud routing.
- Domain constraints determine architectural suitability; no single hybrid or multi-agent design is consistently best across applications.

4.6 Cross-RQ Synthesis

Viewed together, the four research questions suggest a consistent systems principle: hybrid language-model design works best when the large model is treated as a selectively invoked capability rather than the default engine for every token, query, or subtask.

This conclusion is supported by the outcome coding performed in the third pass (Section ??): cross-tabulating performance outcome codes with orchestration pattern codes reveals that studies classified as achieving LLM-competitive or superior results (39 of the 54 studies) consistently feature at least one explicit control layer. To sharpen the directionality of this association: all 39 high-performing studies include a control layer, while the 15 studies with mixed or negative results most frequently involve orchestration overheads—poorly calibrated routing thresholds, high-latency retrieval pipelines, or multi-agent cycles with insufficient stopping criteria—that are not offset by performance gains. A control layer is therefore a necessary but not sufficient condition for strong results; its calibration and fit to the task structure matter as much as its presence. Strong results are reliably associated with explicit control layers such as routers, planners, retrievers, or fusion modules; the absence of such a layer is the most consistent differentiator between high- and low-performing collaborative designs.

Future progress is therefore unlikely to come only from making SLMs slightly larger or LLMs slightly cheaper; it will also depend on better confidence estimation, task decomposition, retrieval selection, and cross-model scheduling.

Methodologically, because accuracy dominates reporting, many studies overstate practical efficiency. The review includes cases where orchestration clearly reduces cost or latency, but also cases where coordination merely shifts the bottleneck. A mature evaluation protocol should therefore report resource metrics as first-class outcomes. Application-wise, the strongest SLM-centered deployments are domain-adapted systems in which the small model is grounded, specialized, or protected by escalation logic. This is why telecom question answering, selected healthcare tasks, optimisation, and industrial monitoring are prominent in the positive findings.

4.7 Implications for Practitioners

For software engineers and system architects, the main design trade-offs are operational rather than purely model-centric. In practice, latency, cost, privacy, and hardware constraints often dominate deployment decisions. Four practical design patterns were identified through frequency analysis of the architecture and orchestration coding results (Section **??**); each pattern is supported by at least four primary studies reporting consistent positive outcomes under similar motivating constraints (privacy sensitivity, throughput requirements, task structure, or data modality), and the supporting study sets are listed below.

1. **Local SLM + RAG** (privacy-constrained settings): When strict data privacy is the main constraint, local SLM + RAG architectures are most suitable for healthcare, telecom, and defense settings (S17, S23, S19, S30), but their performance depends heavily on retriever quality.
2. **Hybrid edge–cloud with routing** (high-throughput settings): When high throughput is required, hybrid edge–cloud architectures are appropriate for IoT and network traffic scenarios (S37, S40, S48, S49), reducing API costs while requiring careful threshold tuning.
3. **Multi-agent SLM with role decomposition** (logical reasoning tasks): For logical reasoning tasks such as code and mathematics, multi-agent SLM designs can improve accuracy (S29, S15, S43, S52), but they introduce significant latency overhead.
4. **Heterogeneous fusion** (multimodal data): For multimodal data, heterogeneous fusion is better suited to medical imaging and robotics (S8, S20, S19, S1), although the slowest model in the pipeline often becomes the bottleneck.

Three practitioner-facing principles follow: (1) **Design for routing**: orchestration, not model size, drives performance; (2) **Balance latency-cost trade-offs**: real-time tasks favor token skipping, while batch tasks tolerate multi-agent loops; and (3) **Treat RAG as control**: retrieval constrains the SLM action space and grounds outputs in verifiable facts.

5 Threats to Validity

5.1 Internal Validity

Although the review follows a PRISMA-guided workflow, study identification, screening, and extraction still involve researcher judgment, especially for borderline papers in which the SLM component or empirical contribution was not equally central. To reduce this risk, borderline screening decisions were discussed through team consensus, and the synthesis remained tied to the predefined research questions and the same extraction matrix throughout.

The review does not apply a formal quality appraisal instrument to individual primary studies, a step recommended by both PRISMA 2020 and the Kitchenham guidelines [?]. This decision reflects the heterogeneity of the corpus: studies span diverse domains, evaluation protocols, and contribution types, making a single quality scale difficult to apply uniformly without introducing subjective scoring variance. Instead, quality-relevant information—reported limitations, baseline availability, and metric completeness—was captured in the extraction matrix (Table ??) and used to modulate synthesis claims. Specifically, claims are explicitly caveated when the supporting study set lacks a reported baseline or when the primary metric is accuracy alone.

5.2 Construct Validity

Core constructs such as "SLM", "hybrid architecture", "multi-agent system", and "efficiency" are not standardized across the literature. A single hybrid label may refer to token-level verification, retrieval-grounded generation, planner-based decomposition, or multimodal fusion, while efficiency may mean latency, FLOPs, API cost, bandwidth reduction, or energy use. This limits direct comparability because studies grouped under the same high-level category may differ substantially in mechanism. To partially limit this, the review separates broad architecture labels from extracted orchestration mechanisms and discusses recurring mechanism families explicitly.

5.3 Conclusion Validity

Cross-study inference is limited by the uneven and inconsistent reporting of evaluation outcomes. Accuracy is reported in most papers, but latency, cost, and energy appear only in a minority, often with different units, hardware assumptions, or deployment settings, which prevents formal meta-analysis and limits claims about efficiency gains across the included studies. Publication and reporting bias may further favor positive hybrid results, so the extraction also recorded reported limitations, and the synthesis emphasizes uneven reporting and task-sensitive trade-offs rather than universal gains.

5.4 External Validity

The study set is based on a Web of Science-centered search and a bounded review window, which improves transparency but may miss relevant studies from other databases, preprint platforms, or fast-moving venues. Although WoS covers many major peer-reviewed journals and conference proceedings, the field evolves quickly and the current corpus is heavily concentrated in 2025 and in a limited set of application domains. Therefore, the conclusions should be generalized cautiously beyond the covered time period, tasks, and deployment settings.

6 Conclusion

This paper presented a systematic literature review of 54 studies on collaborative SLM–LLM systems across hybrid, multi-agent, ensemble, and retrieval-centered architectures, with monolithic LLMs treated as baselines rather than as a primary study class. The four research questions yield consistent, actionable answers.

RQ1 (Performance). Hybrid and multi-agent architectures consistently outperform monolithic SLM baselines and can match or exceed LLM baselines in narrower, structured settings. Performance gains are driven by orchestration strategy—selective escalation, role decomposition, verification loops—rather than by model size alone, and are strongest in domain-specific tasks with well-defined subtask boundaries.

RQ2 (Efficiency). The strongest efficiency gains, often exceeding 80% cost reduction, arise from token-level routing and selective escalation. Latency effects are polarized: local SLM inference is fast, but orchestration layers frequently introduce overhead that negates throughput gains. Energy remains the least reported metric; available evidence shows that generation length and reasoning depth matter as much as model size for per-query energy use, and pushing small models into long Chain-of-Thought traces can exceed the per-query energy cost of a larger model prompted more directly [?].

RQ3 (Orchestration). Orchestration mechanisms cluster into five recurring families: token-level verification, query-level routing, RAG, task decomposition, and multimodal fusion. The choice among them is determined primarily by deployment constraints and data modalities rather than by raw model capacity.

RQ4 (Domain Suitability). Domain-specific constraints—privacy mandates in healthcare, latency budgets in telecom, hardware limits at the edge, and syntax rigidity in code tasks—are the primary drivers of architecture selection. Collaborative SLM–LLM systems deliver their strongest results when the workflow can be partitioned to align each component with the constraint it best addresses.

Across all four questions, a consistent principle emerges: the presence of an explicit control layer—router, planner, retriever, or fusion module—is the most reliable differentiator between high- and low-performing collaborative designs (39 of 54 included studies). Strong results are reliably associated with these control layers rather than with model scale alone.

Design guidelines. The following guidelines represent the most reliable actionable conclusions of this review, each supported by convergent evidence from at least four independent primary studies reporting consistent positive outcomes under comparable operational constraints.

1. **G1 — Prioritise explicit control layers.** When choosing between monolithic and collaborative designs, deploy a routing, retrieval, or planning component as the primary architectural decision; 39 of 54 studies attribute performance gains to the presence of such a layer rather than to model scale alone.
2. **G2 — Match mechanism to deployment constraint.** Use token-level routing when API cost or latency is the binding constraint; use RAG when hallucination reduction or data privacy is primary; use role-specialised multi-agent decomposition when reasoning depth across subtasks is required. Figure **??** provides a practitioner-facing decision procedure.
3. **G3 — Report latency and cost alongside accuracy.** Accuracy-only evaluation is insufficient to establish deployment suitability; efficiency trade-offs are equally decision-relevant and currently underreported across the corpus.
4. **G4 — Prefer narrow, structured domains for initial hybrid deployment.** Collaborative SLM–LLM systems show their most consistent gains in healthcare, telecom, edge/IoT, and code tasks with well-defined subtask boundaries; performance in broad, open-ended generation tasks is less reliable and should be validated against a monolithic baseline before adoption.

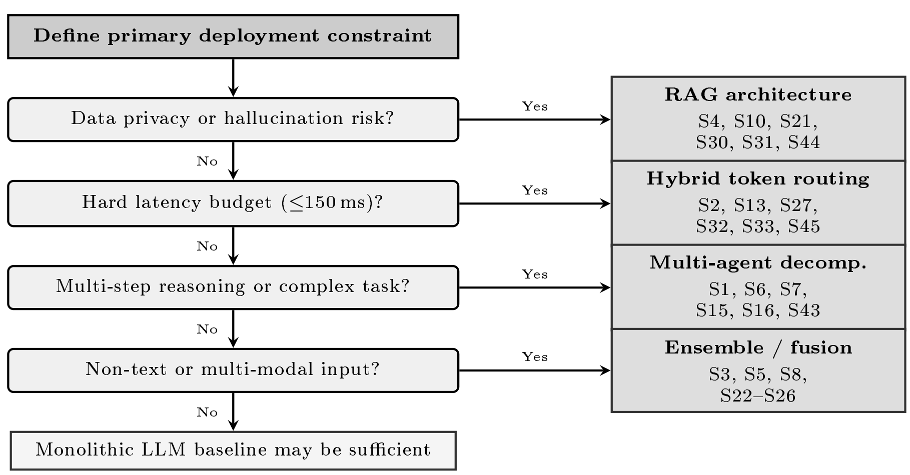

Fig. 4. Architecture selection flowchart derived from corpus synthesis (Guideline G2). Study identifiers refer to Appendix **??**.

The literature is stronger in architectural innovation than in evaluation discipline. Many studies report accuracy improvements, but far fewer report latency, cost, or energy in a form that supports fair comparison. The current evidence is sufficient to show that hybridization *can* work, but often insufficient to show *when* it is the better deployment choice under realistic resource constraints. The strongest positive results remain concentrated in narrower or more structured settings such as telecom question answering, selected healthcare tasks, optimisation, and industrial monitoring. Energy evidence is especially limited: direct measurement is rare, and proxy improvements such as reduced computational redundancy [**?**] do not establish verified energy savings. Future work should treat energy as a first-class evaluation outcome and report energy-per-token, reasoning length, hardware configuration, and escalation behavior under comparable conditions. Standardized benchmarks for collaborative SLM–LLM systems would further reduce the current reliance on paper-specific evaluation protocols.

The full comparative analysis table and data extraction spreadsheet supporting this review are available at `https://bit.ly/SLM-LLM-Data-2026`.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

A Online Appendix

The online appendix contains the full comparative analysis table for the 54 primary studies and the comprehensive data extraction spreadsheet used in this review.
Access link: `https://bit.ly/SLM-LLM-Data-2026`

B Primary Study Identifier Mapping

Table **??** maps the S1–S54 study identifiers used throughout the synthesis to their bibliographic references.

References

1. Esteban-Romero, S., Martín-Fernández, I., Gil-Martín, M., Fernández-Martínez, F.: Synthesizing Olfactory Understanding: Multimodal Language Models for Image-Text Smell Matching. Symmetry **17**(8), 1349 (2025). doi:10.3390/sym17081349
2. Shivarpatna Venkatesh, A.P., Sunil, R., Sabu, S., Reis, M.S., Bodden, E., Amir, M.: An Empirical Study of Large Language Models for Type and Call Graph Analysis in Python and JavaScript. arXiv preprint arXiv:2410.00603 (2025). doi:10.48550/arXiv.2410.00603
3. Chen, H., Deng, W., Yang, S., Xu, J., Jiang, Z., Ngai, E.C.H., et al.: Toward Edge General Intelligence via Large Language Models: Opportunities and Challenges. arXiv preprint arXiv:2410.18125 (2025). doi:10.48550/arXiv.2410.18125

Table 6. Study identifier to citation mapping (S1–S54).

ID	Ref.	ID	Ref.	ID	Ref.
S1	[?]	S19	[?]	S37	[?]
S2	[?]	S20	[?]	S38	[?]
S3	[?]	S21	[?]	S39	[?]
S4	[?]	S22	[?]	S40	[?]
S5	[?]	S23	[?]	S41	[?]
S6	[?]	S24	[?]	S42	[?]
S7	[?]	S25	[?]	S43	[?]
S8	[?]	S26	[?]	S44	[?]
S9	[?]	S27	[?]	S45	[?]
S10	[?]	S28	[?]	S46	[?]
S11	[?]	S29	[?]	S47	[?]
S12	[?]	S30	[?]	S48	[?]
S13	[?]	S31	[?]	S49	[?]
S14	[?]	S32	[?]	S50	[?]
S15	[?]	S33	[?]	S51	[?]
S16	[?]	S34	[?]	S52	[?]
S17	[?]	S35	[?]	S53	[?]
S18	[?]	S36	[?]	S54	[?]

4. Wilhelm, P., Wittkopp, T., Kao, O.: Beyond Test-Time Compute Strategies: Advocating Energy-per-Token in LLM Inference. In: Proceedings of the 5th Workshop on Machine Learning and Systems (EuroMLSys 2025), pp. 208–215. ACM, Rotterdam, The Netherlands (2025). doi:10.1145/3721146.3721953
5. Bao, R., Xue, N., Sun, Y., Chen, Z.: Dynamic Quality-Latency Aware Routing for LLM Inference in Wireless Edge-Device Networks. arXiv preprint arXiv:2508.11291 (2025). doi:10.48550/arXiv.2508.11291
6. Tigani, S.: CPU-Only Self Enhancing Authoring Copilot Design-Based Markov Decision Processes Orchestration and Qwen 3 Local Large Language Model. Technologies **13**(11), 520 (2025). doi:10.3390/technologies13110520
7. Aydın, S.K., Ali, R.H., Faiz, S., Khan, T.A.: An Integrated AI Framework for Personalized Nutrition Using Machine Learning and Natural Language Processing for Dietary Recommendations. Appl. Sci. **15**(17), 9283 (2025). doi:10.3390/app15179283
8. Xie, L., Jin, Y., Xu, L., Chang, S., Xu, X.: Fusing Domain Knowledge with a Fine-Tuned Large Language Model for Enhanced Molecular Property Prediction. J. Chem. Theory Comput. **21**(14), 6743–6758 (2025). doi:10.1021/acs.jctc.5c00605
9. Zhu, X., Li, J., Liu, Y., Ma, C., Wang, W.: Distilling Mathematical Reasoning Capabilities into Small Language Models. arXiv preprint arXiv:2401.11864 (2024). doi:10.48550/arXiv.2401.11864
10. Alabbasi, N., Alhussein, O., Muhaidat, S., Debbah, M.: Do you actually need an LLM? Rethinking language models for customer reviews analysis. arXiv preprint arXiv:2411.02555 (2024). doi:10.21203/rs.3.rs-5006214/v1
11. Ji, X., Chen, S., Shon, Y., et al.: RAP-RAG: A Retrieval-Augmented Generation Framework with Adaptive Retrieval Task Planning. Electronics **14**(1), 185 (2025). 10.3390/electronics14214269
12. Dorfner, F.J., Gadewar, M.R., et al.: Comparing Commercial and Open-Source Large Language Models for Labeling Chest Radiograph Reports. Radiology **313**(1), e240590 (2024). doi:10.1148/radiol.241139

13. Hao, Z., Wei, J., Wang, T., Ren, J.: Hybrid SLM and LLM for Edge-Cloud Collaborative Inference. In: Proceedings of the 2024 Workshop on Edge and Mobile Foundation Models (EdgeFM 2024), pp. 36–41. ACM, Tokyo, Japan (2024). doi:10.1145/3662006.3662067
14. Shao, C., Hu, X., Lin, Y., Xu, F.: Division-of-Thoughts: Harnessing Hybrid Language Model Synergy for Efficient On-Device Agents. In: Proceedings of the ACM Web Conference 2025 (WWW 2025), pp. 1822–1833. ACM, Sydney, Australia (2025). doi:10.1145/3696410.3714765
15. Zeng, J., Liao, J., Sun, S., Zhang, G., Xu, C., Liu, H., et al.: Multi-Agent Collaborative Reasoning via Cloud-Edge Framework. IEEE Network (2026). doi:10.1109/MNET.2026.3651036
16. Lee, C.H., Cheng, H., Ostendorf, M.: OrchestraLLM: Efficient Orchestration of Language Models for Dialogue State Tracking. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 1434–1445. Association for Computational Linguistics, Mexico City, Mexico (2024). doi:10.18653/v1/2024.naacl-long.79
17. Alabbasi, N., Erak, O., Alhussein, O., Lotfi, I., Muhaidat, S., Debbah, M.: TeleOracle: Fine-Tuned Retrieval-Augmented Generation with Long-Context Support for Networks. arXiv preprint arXiv:2411.02617 (2024). doi:10.48550/arXiv.2411.02617
18. Chatzistefanidis, I., Nikaein, N.: Symbiotic Agents: A Novel Paradigm for Trustworthy AGI-driven Networks. Comput. Networks **273**, 111749 (2025). doi:10.1016/j.comnet.2025.111749
19. Han, J., Kim, M., Kim, Y., Lee, W.H.: Self-Reflective Chest X-Ray Report Generation with Clinical-Aware Detection and Multilevel Readability. J. Med. Syst. **49**, 190 (2025). doi:10.1007/s10916-025-02326-5
20. Corradini, F., Leonesi, M., Piangerelli, M.: State of the Art and Future Directions of Small Language Models: A Systematic Review. Big Data Cogn. Comput. **9**(7), 189 (2025). doi:10.3390/bdcc9070189
21. Wang, F., et al.: A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. arXiv preprint arXiv:2411.03350 (2024). doi:10.48550/arXiv.2411.03350
22. Chen, Y., et al.: A Survey on Collaborative Mechanisms Between Large and Small Language Models. arXiv preprint arXiv:2505.07460 (2025). doi:10.48550/arXiv.2505.07460
23. Wang, L., et al.: A Survey on Collaborating Small and Large Language Models for Performance, Cost-effectiveness, Cloud-edge Privacy, and Trustworthiness. arXiv preprint arXiv:2510.13890 (2025). doi:10.48550/arXiv.2510.13890
24. Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., et al.: The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews. BMJ **372**, n71 (2021). doi:10.1136/bmj.n71
25. Kitchenham, B., Charters, S.: Guidelines for Performing Systematic Literature Reviews in Software Engineering. Technical Report EBSE-2007-01, Keele University and Durham University Joint Report (2007)
26. Petersen, K., Feldt, R., Mujtaba, S., Mattsson, M.: Systematic Mapping Studies in Software Engineering. In: Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering (EASE 2008), pp. 68–77. BCS Learning and Development Ltd., Swindon, UK (2008)
27. Webster, J., Watson, R.T.: Analyzing the Past to Prepare for the Future: Writing a Literature Review. MIS Q. **26**(2), xiii–xxiii (2002)

28. Cao, Q., et al.: Toward Energy-Safe Industrial Monitoring: A Hybrid LLM Framework for Video Captioning. Appl. Sci. **15**(23), 12848 (2025). doi:10.3390/app152312848
29. Han, X., Sun, J., Yu, S., Qin, L., Luo, H., Li, Y.: CommentAgent: a LLM-Powered Agent Framework for Automated Comment Generation and Opinion Understanding. J. Supercomput. **81**(15), 1438 (2025). doi:10.1007/s11227-025-07957-6
30. Demir, E., Bilgin, M.: DPATransLLM: Detection of Pronominal Anaphora in Turkish Sentences Using Transformer-Based, Large Language Models and Hybrid Ensemble Approach. Appl. Sci. **15**(23), 12480 (2025). doi:10.3390/app152312480
31. Pacheco De Almeida Prado, J., Zammit Mangion, D., Zammit, B., Gauci, J., Muscat, A., Mizzi, S., et al.: Enhancing Flight Deck Decision Support with Distributed GenAI: A Multi-Agent Approach. In: Proceedings of the 2025 IEEE Aerospace Conference (AERO), pp. 1–20. IEEE, Big Sky, MT, USA (2025). doi:10.1109/AERO63441.2025.11068478
32. Zhu, J., Wang, L., Guc, Y., Wang, F.: OFSLMs: Offline Fine-Tuned Small Language Models Based on Hybrid Synthetic Knowledge for Robot Introspective Decision-Making. In: Proceedings of the 2024 IEEE International Conference on Robotics and Biomimetics (ROBIO), pp. 1839–1844. IEEE, Bangkok, Thailand (2024). doi:10.1109/ROBIO64047.2024.10907647
33. Jeong, S., Baek, J., Cho, S., Hwang, S., Park, J.: Test-Time Self-Adaptive Small Language Models for Question Answering. In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 15459–15469. Association for Computational Linguistics, Singapore (2023). doi:10.18653/v1/2023.findings-emnlp.1033
34. Weiss, Y., Buttery, P., Africa, D.D., Diehl Martinez, R.: Investigating ReLoRA: Effects on the Learning Dynamics of Small Language Models. arXiv preprint arXiv:2509.12960 (2025). doi:10.48550/arXiv.2509.12960
35. Pietrzak, Z., Mączka, K., Niemiec, M.: What if, Behind the Curtain, There Is Only an LLM? A Holistic Evaluation of TinyLlama-Generated Synthetic Cyber Threat Intelligence. Electronics **14**(24), 4971 (2025). doi:10.3390/electronics14244971
36. Gao, Y., Myers, S., Chen, S., Dligach, D., Miller, T., Bitterman, D.S., et al.: Uncertainty Estimation in Diagnosis Generation from Large Language Models: Next-Word Probability Is Not Pre-Test Probability. JAMIA Open **8**(1), ooae154 (2025). doi:10.1093/jamiaopen/ooae154
37. Xiong, Y., Huang, M., Liang, X., Tao, M.: ResilioMate: A Resilient Multi-Agent Task Executing Framework for Enhancing Small Language Models. IEEE Access **13**, 86892–86911 (2025). doi:10.1109/ACCESS.2025.3567244
38. Khader, K.A., Hussein, M.S., Abu-Issa, A.S.: Adapting Large Language Models for Arabic: Comparative Evaluation, Fine-Tuning, and Ethical Deployment. IEEE Access **13**, 182621–182632 (2025). doi:10.1109/ACCESS.2025.3623796
39. Kuligin, L., Lammert, J., Ostapenko, A., Bressem, K., Boeker, M., Tschochohei, M.: Prompt Design for Medical Question Answering with Large Language Models. Mach. Learn. Appl. **22**, 100758 (2025). doi:10.1016/j.mlwa.2025.100758
40. Abbas, N., Atwell, E.: Cognitive Computing with Large Language Models for Student Assessment Feedback. Big Data Cogn. Comput. **9**(5), 112 (2025). doi:10.3390/bdcc9050112
41. Wang, Z., Li, K., Peng, S., Liu, L., Yang, X., Yao, K., et al.: A Weighted Voting Approach for Traditional Chinese Medicine Formula Classification Using Large Language Models: Algorithm Development and Validation Study. JMIR Med. Inform. **13**, e69286 (2025). doi:10.2196/69286

42. Dodson, R.J.B., Dodson, C.R.S., Dodson, P.S., et al.: Explainable Machine Learning-Based Prediction of Psoriatic Arthritis Flares Using Heterogenous Real-World Data for Personalised Patient Care. Methods **246**, 95–106 (2026). doi:10.1016/j.ymeth.2025.10.010
43. Deng, J., Wu, Y., Yesha, Y., Nguyen, P.: Improving VTE Identification through Language Models from Radiology Reports: A Comparative Study of Mamba, Phi-3 Mini, and BERT. arXiv preprint arXiv:2408.09043 (2024). doi:10.48550/arXiv.2408.09043
44. Das, S., Srihari, R.: Compos Mentis at SemEval2024 Task6: A Multi-Faceted Role-Based Large Language Model Ensemble to Detect Hallucination. In: Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024), pp. 1449–1454. Association for Computational Linguistics, Mexico City, Mexico (2024). doi:10.18653/v1/2024.semeval-1.208
45. Xu, X., Zhao, J., Zhang, Y., Li, R.: Integrating Reinforcement Learning and LLM with Self-Optimization Network System. Network **5**(3), 39 (2025). doi:10.3390/network5030039
46. Erak, O., Alabbasi, N., Alhussein, O., Lotfi, I., Hussein, A., Muhaidat, S., et al.: Leveraging Fine-Tuned Retrieval-Augmented Generation with Long-Context Support: For 3GPP Standards. arXiv preprint arXiv:2408.11775 (2025). doi:10.48550/arXiv.2408.11775
47. Afzal, A., Vladika, J., Fazlija, G., Staradubets, A., Matthes, F.: Towards Optimizing a Retrieval Augmented Generation using Large Language Model on Academic Data. In: Proceedings of the 2024 8th International Conference on Natural Language Processing and Information Retrieval (NLPIR 2024), pp. 250–257. ACM, Okayama, Japan (2025). doi:10.1145/3711542.3711575
48. Solat, F., Lee, J., Seif, M., Niyato, D., Poor, H.V.: Federated Learning-Enabled Hybrid Language Models for Communication-Efficient Token Transmission. IEEE Internet Things J. **12**(24), 53574–53592 (2025). doi:10.1109/JIOT.2025.3618095
49. Lee, C., Kumar, S., Vogt, K.A., Munshi, M., Tallapudi, P., Vogt, A., et al.: Democratizing Cost-Effective, Agentic Artificial Intelligence to Multilingual Medical Summarization Through Knowledge Distillation. Sci. Rep. **15**, 27619 (2025). doi:10.1038/s41598-025-10451-x
50. Jin, X., Wang, Z., Chen, J., Yang, L., Oh, B., Hwang, S., et al.: HLMEA: Unsupervised Entity Alignment Based on Hybrid Language Models. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, no. 11, pp. 11888–11896. AAAI Press, Philadelphia, PA, USA (2025). doi:10.1609/aaai.v39i11.33294
51. Huynh, T., Nguyen, D.: Collaborating Agents with Large Language Models for Question-Answering Tasks. In: Proceedings of the 5th IEEE International Conference on Software Engineering and Artificial Intelligence (SEAI), pp. 1–5. IEEE, Fuzhou, China (2025). doi:10.1109/SEAI65851.2025.11108177
52. Al Daoud, E., Al Daoud, L., Asassfeh, M., Al-Shaikh, A., Al-Sherideh, A.S., Afaneh, S.: Enhancing Cybersecurity with Transformers: Preventing Phishing Emails and Social Media Scams. In: Proceedings of the 2024 IEEE Conference on Dependable and Secure Computing (DSC), pp. 31–36. IEEE, Tokyo, Japan (2024). doi:10.1109/DSC63325.2024.00017
53. Ying, S., Li, Z., Yu, M.: Beyond Words: Evaluating Large Language Models in Transportation Planning. arXiv preprint arXiv:2409.14516 (2024). doi:10.48550/arXiv.2409.14516
54. Pintelas, E., Koursaris, A., Livieris, I.E., Tampakas, V.: TextNeX: Text Network of eXperts for Robust Text Classification-Case Study on Machine-Generated-Text Detection. Mathematics **13**(10), 1555 (2025). doi:10.3390/math13101555

55. Gichamba, A., Idris, T.K., Ebiyau, B., Nyberg, E., Mitamura, T.: ColBERT Retrieval and Ensemble Response Scoring for Language Model Question Answering. arXiv preprint arXiv:2408.10808 (2024). doi:10.48550/arXiv.2408.10808
56. Oh, S., Kim, J., Park, J., Ko, S.W., Quek, T.Q.S., Kim, S.L.: Uncertainty-Aware Hybrid Inference with On-Device Small and Remote Large Language Models. In: Proceedings of the 2025 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN), pp. 1–7. IEEE, Barcelona, Spain (2025). doi:10.1109/ICMLCN64995.2025.11140540
57. She, J., Zheng, W., Liu, Z., Wang, H., Xing, E.P., Yao, H., et al.: Token Level Routing Inference System for Edge Devices. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), pp. 159–166. Association for Computational Linguistics, Vienna, Austria (2025). doi:10.18653/v1/2025.acl-demo.16
58. Cielen, D., De Bock, K.W., Florès, L.: Exploring Zero-Shot SLM Ensembles as an Alternative to LLMs for Sentiment Analysis. Inf. Fusion **126**, 103666 (2026). doi:10.1016/j.inffus.2025.103666
59. Li, M., Jiao, H., Zhou, T., Zhang, N., Peters, S., Lissitz, R.W.: Item Difficulty Modeling Using Fine-Tuned Small and Large Language Models. Educ. Psychol. Meas. **85**(6), 1065–1090 (2025). doi:10.1177/00131644251344973
60. Mehandru, N., Hall, A.K., Melnichenko, O., Dubinina, Y., Tsirulnikov, D., Bamman, D., et al.: BioAgents: Bridging the Gap in Bioinformatics Analysis with Multi-Agent Systems. Sci. Rep. **15**, 39036 (2025). doi:10.1038/s41598-025-25919-z
61. Amarasinghe, P.T., Nguyen, S., Sun, Y., Alahakoon, D.: Feedback-Integrated Prompt Optimiser for Problem Formulation. Sci. Rep. **15**, 43543 (2025). doi:10.1038/s41598-025-27495-8
62. Li, C., Chen, Q., Li, L., Wang, C., Li, Y., Chen, Z., et al.: Mixed Distillation Helps Smaller Language Model Better Reasoning. arXiv preprint arXiv:2312.10730 (2024). doi:10.48550/arXiv.2312.10730